

Q1 Multilayer Perceptron (MLP) using backpropagation

```
# =====
# Multilayer Perceptron (MLP) using Backpropagation
# Binary Classification Example
# =====

import numpy as np
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report

# -----
# Step 1: Generate Binary Classification Dataset
# -----
X, y = make_classification(
    n_samples=500,
    n_features=4,
    n_informative=4,
    n_redundant=0,
    n_classes=2,
    random_state=42
)

# Split into Training and Testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# -----
# Step 2: Create Multilayer Perceptron
# hidden_layer_sizes=(5,) means one hidden layer with 5 neurons
# -----
mlp = MLPClassifier(
    hidden_layer_sizes=(5,),
    activation='relu',
    solver='sgd',
    learning_rate_init=0.01,
    max_iter=1000,
    random_state=42
)

# -----
# Step 3: Train the Model
# Backpropagation automatically updates the weights
# -----
mlp.fit(X_train, y_train)

# -----
# Step 4: Predict
# -----
y_pred = mlp.predict(X_test)

# -----
# Step 5: Evaluate
# -----
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

print("\nConfusion Matrix")
print(confusion_matrix(y_test, y_pred))

print("\nClassification Report")
print(classification_report(y_test, y_pred))

# -----
# Display Learned Weights
# -----
print("\nInput -> Hidden Layer Weights")
print(mlp.coefs_[0])
```

```
print("\nHidden -> Output Layer Weights")
print(mlp.coefs_[1])

print("\nTraining Loss")
print(mlp.loss_)
```

Accuracy: 0.87

Confusion Matrix

```
[[47  7]
 [ 6 40]]
```

Classification Report

	precision	recall	f1-score	support
0	0.89	0.87	0.88	54
1	0.85	0.87	0.86	46
accuracy			0.87	100
macro avg	0.87	0.87	0.87	100
weighted avg	0.87	0.87	0.87	100

Input -> Hidden Layer Weights

```
[[-0.52317912  0.98507495  0.68214536  0.04592223 -1.28235795]
 [-0.86251265 -1.33787007  0.89867303  1.0320642  0.36248013]
 [ 0.18711459  0.18223439  0.28815806 -0.89937655 -0.59255977]
 [-1.08429654 -0.34870703  0.05293656  1.081401  0.24238175]]
```

Hidden -> Output Layer Weights

```
[[ 1.14504198]
 [-1.61188673]
 [ 0.67328357]
 [ 1.69029754]
 [-1.48930857]]
```

Training Loss

0.3293437185620234

Q2

```
import random

# -----
# Initial Population
# -----
population = [
    {"name": "A", "speed": 40, "accuracy": 60},
    {"name": "B", "speed": 50, "accuracy": 65},
    {"name": "C", "speed": 60, "accuracy": 70},
    {"name": "D", "speed": 70, "accuracy": 75},
    {"name": "E", "speed": 80, "accuracy": 80},
    {"name": "F", "speed": 90, "accuracy": 85}
]

# -----
# Fitness Function
# -----
def fitness(individual):
    return (0.4*individual["speed"] +
           0.6*individual["accuracy"])

# -----
# Display Population
# -----
def show_population(pop, title):

    print("\n", title)
    print("-"*60)

    for p in pop:

        fit=fitness(p)

        suitable="Yes" if fit>=70 else "No"

        print(f"{p['name']} Speed={p['speed']} "
              f"Accuracy={p['accuracy']} ")
```

```

        f"Fitness={fit:.2f} Suitable={suitable}")

# -----
# Selection (Top 2)
# -----
def selection(pop):

    pop=sorted(pop,key=fitness,reverse=True)

    return pop[:2]

# -----
# Crossover
# -----
def crossover(parent1,parent2):

    child={}

    child["name"]="Child"

    child["speed"]=parent1["speed"]

    child["accuracy"]=parent2["accuracy"]

    return child

# -----
# Mutation
# -----
def mutation(child):

    child["speed"]+=random.randint(-5,5)

    child["accuracy"]+=random.randint(-5,5)

    child["speed"]=max(0,min(100,child["speed"]))
    child["accuracy"]=max(0,min(100,child["accuracy"]))

    return child

# -----
# Generation 0
# -----
show_population(population,"Generation 0")

# -----
# Selection
# -----
parents=selection(population)

print("\nSelected Parents")

for p in parents:

    print(p)

# -----
# Crossover
# -----
child=crossover(parents[0],parents[1])

print("\nAfter Crossover")

print(child)

# -----
# Mutation
# -----
child=mutation(child)

print("\nAfter Mutation")

print(child)

population.append(child)

# -----

```

```
# Final Population
# -----
show_population(population,"Generation 1")

best=max(population,key=fitness)

print("\nBest Optimal Solution")

print(best)

print("Fitness =",fitness(best))
```

```
Generation 0
-----
A Speed=40 Accuracy=60 Fitness=52.00 Suitable=No
B Speed=50 Accuracy=65 Fitness=59.00 Suitable=No
C Speed=60 Accuracy=70 Fitness=66.00 Suitable=No
D Speed=70 Accuracy=75 Fitness=73.00 Suitable=Yes
E Speed=80 Accuracy=80 Fitness=80.00 Suitable=Yes
F Speed=90 Accuracy=85 Fitness=87.00 Suitable=Yes

Selected Parents
{'name': 'F', 'speed': 90, 'accuracy': 85}
{'name': 'E', 'speed': 80, 'accuracy': 80}

After Crossover
{'name': 'Child', 'speed': 90, 'accuracy': 80}

After Mutation
{'name': 'Child', 'speed': 91, 'accuracy': 76}

Generation 1
-----
A Speed=40 Accuracy=60 Fitness=52.00 Suitable=No
B Speed=50 Accuracy=65 Fitness=59.00 Suitable=No
C Speed=60 Accuracy=70 Fitness=66.00 Suitable=No
D Speed=70 Accuracy=75 Fitness=73.00 Suitable=Yes
E Speed=80 Accuracy=80 Fitness=80.00 Suitable=Yes
F Speed=90 Accuracy=85 Fitness=87.00 Suitable=Yes
Child Speed=91 Accuracy=76 Fitness=82.00 Suitable=Yes

Best Optimal Solution
{'name': 'F', 'speed': 90, 'accuracy': 85}
Fitness = 87.0
```

Q3

```
# =====
# Neural Network Classification using Backpropagation
# =====

import pandas as pd
from sklearn.preprocessing import LabelEncoder
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
)

# -----
# Step 1: Create Dataset
# -----

data = {
    "Age": [45, 50, 30, 60, 40],
    "BloodPressure": ["High", "Normal", "Normal", "High", "Normal"],
    "GlucoseLevel": ["High", "High", "Normal", "High", "Normal"],
    "LifestyleRisk": ["High", "Medium", "Low", "High", "Medium"],
    "Actual": ["Yes", "Yes", "No", "Yes", "No"]
}

df = pd.DataFrame(data)

print("Dataset")
print(df)
```

```

# -----
# Step 2: Encode Categorical Features
# -----

encoder = LabelEncoder()

df["BloodPressure"] = encoder.fit_transform(df["BloodPressure"])
df["GlucoseLevel"] = encoder.fit_transform(df["GlucoseLevel"])
df["LifestyleRisk"] = encoder.fit_transform(df["LifestyleRisk"])
df["Actual"] = encoder.fit_transform(df["Actual"])

# -----
# Step 3: Features and Target
# -----

X = df.drop("Actual", axis=1)
y = df["Actual"]

# -----
# Step 4: Train Neural Network
# -----

model = MLPClassifier(
    hidden_layer_sizes=(4,),
    activation='relu',
    solver='lbfgs',
    max_iter=1000,
    random_state=42
)

model.fit(X, y)

# -----
# Step 5: Prediction
# -----

predicted = model.predict(X)

print("\nPredicted Output")

print(predicted)

# -----
# Step 6: Confusion Matrix
# -----

cm = confusion_matrix(y, predicted)

TN, FP, FN, TP = cm.ravel()

print("\nConfusion Matrix")
print(cm)

print("\nTrue Positive :", TP)
print("True Negative :", TN)
print("False Positive:", FP)
print("False Negative:", FN)

# -----
# Step 7: Evaluation Metrics
# -----

print("\nAccuracy :", accuracy_score(y, predicted))
print("Precision:", precision_score(y, predicted))
print("Recall   :", recall_score(y, predicted))
print("F1 Score :", f1_score(y, predicted))

```

Dataset

	Age	BloodPressure	GlucoseLevel	LifestyleRisk	Actual
0	45	High	High	High	Yes
1	50	Normal	High	Medium	Yes
2	30	Normal	Normal	Low	No
3	60	High	High	High	Yes
4	40	Normal	Normal	Medium	No

Predicted Output

[1 1 1 1 1]

Confusion Matrix

```
[[0 2]
```

```
[0 3]]
```

True Positive : 3

True Negative : 0

False Positive: 2

False Negative: 0

Accuracy : 0.6

Precision: 0.6

Recall : 1.0

F1 Score : 0.75